Baseline vs Online Join

Physical execution order for hybrid structured and semantic movie search

Ann Remizova

June 17, 2026

Goal

- The task is hybrid movie search over structured IMDb metadata and free-text reviews.
- SUQL gives us SQL plus text functions such as `answer(review, question)` and `summary(review)`.
- The core cost is evaluating semantic review predicates with an LLM.
- This comparison asks: should structured filtering happen before semantic review checks, or should the two branches run independently and join later?

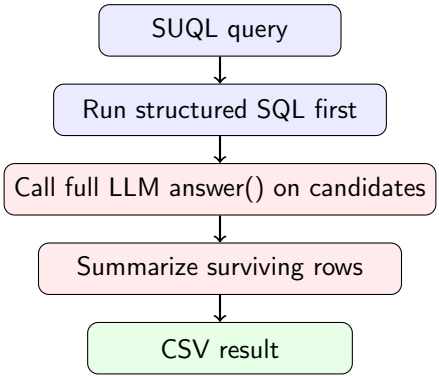
Common Query Shape

Example SUQL query

```
SELECT movie_id, title, year, runtime, director, genres,
 summary(review) AS review_summary
FROM movies
WHERE year >= 1990 AND genres LIKE 'Comedy'
 AND answer(review, 'Does the reviewer say it is funny?')  = 'Yes'
LIMIT 10;
```

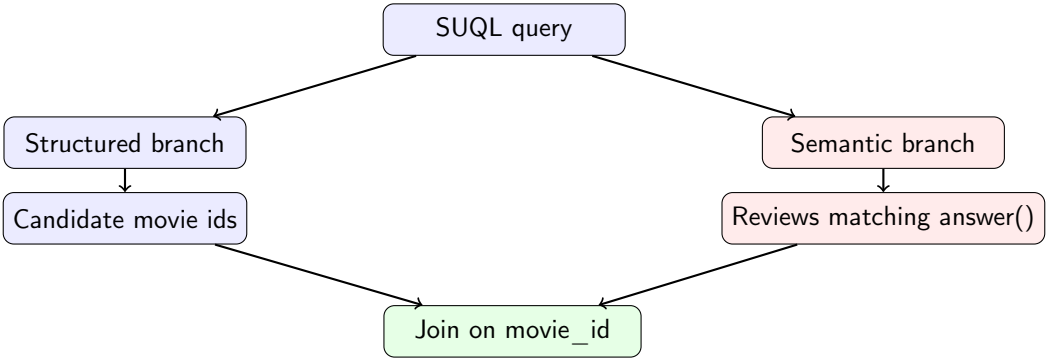
- Structured predicates: `year`, `runtime`, `genres`, `director`.
- Unstructured predicate: `answer(review, ...)`.
- Output enrichment: `summary(review)` for rows that survive filtering.

Baseline: Structured-First Execution



- Structured predicates reduce the review set before any expensive semantic calls.
- Each remaining candidate still uses the full answer model.
- The baseline is simple, but it is already a strong physical plan for selective structured filters.

Online Join: Independent Branches



- Structured and semantic retrieval are evaluated as separate branches.
- The semantic branch scans the review sample before knowing which movie ids survived structured predicates.
- This loses the baseline's main pruning advantage.

Key Difference

Baseline

- SQL filters first.
- LLM sees only structured candidates.
- Cost follows structured selectivity.

Online join

- Semantic scan happens independently.
- LLM sees about the whole review sample per query.
- Cost follows review-sample size.

Practical consequence

Online join can be elegant as a relational plan, but it is expensive when the text side is much larger than the structured candidate set.

Benchmark Provenance

Experiment setup

- Query set: nine fixed SUQL movie-review queries from `Stage_2/benchmark_stage2.py`; each has structured predicates, one `answer(review, ...)` predicate, and `LIMIT 10`.
- Data sample: `Stage_2/benchmarks/data_sample_100/imdb_joined.csv`; online join splits it into 100 review rows and 97 distinct structured movie rows.
- Model: `SUQL_MODEL=ollama/phi4-mini` for baseline and online join full `answer()` / `summary()` calls.
- Metrics: baseline from `Stage_2/benchmarks/gemma2/metrics.csv`; online join from `Stage_2/benchmarks/online_join_vs_stage2/metrics.csv`.

Benchmark Provenance: Query Text

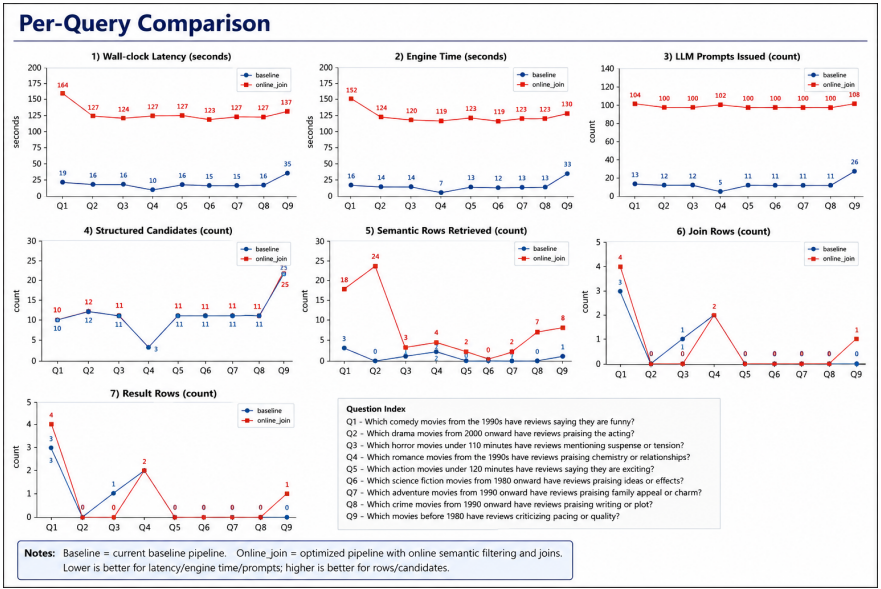
- Q1: Which comedy movies from the 1990s have reviews saying they are funny?
- Q2: Which drama movies from 2000 onward have reviews praising the acting?
- Q3: Which horror movies under 110 minutes have reviews mentioning suspense or tension?
- Q4: Which romance movies from the 1990s have reviews praising chemistry or relationships?
- Q5: Which action movies under 120 minutes have reviews saying they are exciting?
- Q6: Which science fiction movies from 1980 onward have reviews praising ideas or effects?
- Q7: Which adventure movies from 1990 onward have reviews praising family appeal or charm?
- Q8: Which crime movies from 1990 onward have reviews praising writing or plot?
- Q9: Which movies before 1980 have reviews criticizing pacing or quality?

Aggregate Metrics

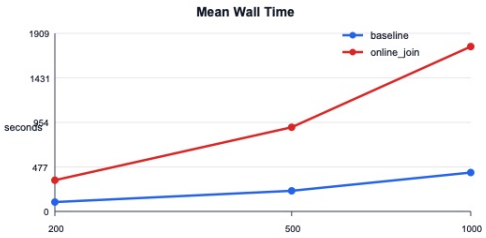
Metric	Baseline	Online join	Change
Wall time	157.40 s	1184.45 s	7.5× slower
Engine time	134.89 s	1132.63 s	8.4× slower
LLM prompts	112	907	8.1× more
Full LLM calls	112	907	8.1× more
Structured candidates	105	101	similar
Semantic rows	7	68	9.7× more
Result rows	7	7	same total

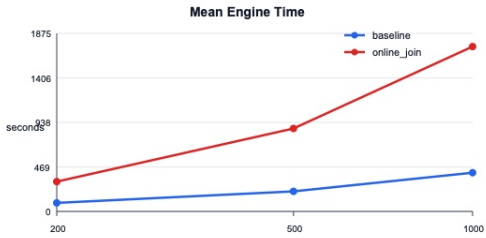
- The final output size is the same, but online join does much more semantic work to get there.
- The major gap is the 907 full answer calls in online join versus 112 in the baseline.

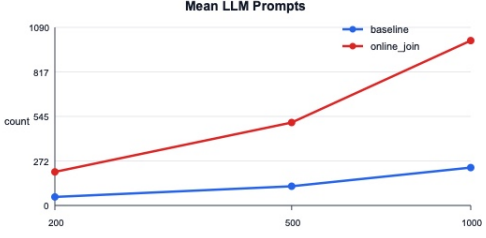
Per-Query Comparison

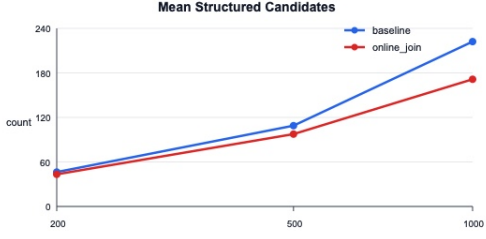


Performance Metrics vs Sample Size









Why Online Join Is Slower Here

- The structured branch is not the problem: both systems retrieve about the same number of structured candidates.
- The semantic branch is the problem: online join evaluates review predicates before structured pruning.
- Most semantic matches are discarded by the later join.
- Runtime follows prompt volume closely because each `answer()` call uses the expensive model.

Main Takeaways

1. **Best of these two plans:** baseline structured-first execution.
2. **Reason:** it applies cheap SQL pruning before expensive semantic review checks.
3. **Online join bottleneck:** independent semantic scans create many full LLM calls.
4. **Observed result:** online join is $7.5\times$ slower wall-clock and uses $8.1\times$ more LLM prompts.
5. **Design lesson:** for hybrid SUQL workloads, the physical plan matters as much as the query syntax.

Future Work

- **Stretto-style planning:** model semantic filters as several physical operators and choose a plan under explicit latency and quality budgets.
- **Denser operator choices:** explore batching, prefix reuse, and KV-cache-aware execution so repeated review predicates are not paid as independent full calls.
- **Potential upgrade:** keep the calibrated accept/reject idea, but make the first-stage scorer genuinely cheaper than full generation.
- **Better routing:** use Stage 1 uncertainty and observed selectivity to decide when rows need the full `answer()` model and when a cheap decision is enough.
- **Plan selection:** combine structured selectivity estimates with semantic-cost estimates so online-style branches are used only when predicted to win.